

# plotting

March 29, 2026

## 1 1. An Introduction To Data Visualization In Python

This book will cover the most relevant and unique attributes and features for 9 different Python libraries, before going on to demonstrate how to visualize data with them. This book will also cover the different types of data you can visualize in Python, in addition to common visualization techniques, tools, and plot types.

Before delving too deeply into the libraries themselves, it would be helpful to gain an intuition of how the landscape of Python's visualization libraries breaks down. To put that another way, it's helpful to understand how the different Python libraries are designed and related to one another. Understanding how the different libraries operate will help you choose the best library for your visualization project.

There are a number of different data visualization libraries and modules compatible with Python. Most of the Python data visualization libraries can be placed into one of four groups, separated based on their origins and focus.

The groups are:

- Matplotlib-based libraries
- JavaScript libraries
- JSON libraries
- WebGL libraries

### 1.1 Matplotlib-based Libraries

The first major group of libraries is those based on Matplotlib. Matplotlib is one of the oldest Python data visualization libraries, and thanks to its wealth of features and ease of use it is still one of the most widely used one. Matplotlib was first released back in 2003 and has been continuously updated since.

Matplotlib contains a large number of visualization tools, plot types, and output types. It produces mainly static visualizations. While the library does have some 3D visualization options, these options are far more limited than those possessed by other libraries like Plotly and VisPy. It is also limited in the field of interactive plots, unlike Bokeh, which we'll cover in a later chapter.

Because of Matplotlib's success as a visualization library, various other libraries have expanded on its core features over the years. These libraries are Matplotlib-based, using Matplotlib as an engine for their own visualization functions.

The libraries based upon Matplotlib add new functionality to the library by specializing in the rendering of certain data types or domains, adding new types of plots, or creating new high-level

APIs for Matplotlib's functions.

They're used *alongside* Matplotlib, not *instead*, to expand its styling and plotting capabilities.

## 1.2 JavaScript-based Libraries

There are a number of JavaScript-based libraries for Python that specialize in data visualization. The adoption of HTML5 by web browsers enabled interactivity for graphs and visualizations, instead of only static 2D plots. Styling HTML pages with CSS can net beautiful visualizations.

These libraries wrap JavaScript/HTML5 functions and tools in Python, allowing the user to create new interactive plots. The libraries provide high-level APIs for the JavaScript functions, and the JavaScript primitives can often be edited to create new types of plots, all from within Python.

## 1.3 JSON-based Libraries

JavaScript Object Notation (JSON) is a data interchange format, containing data in a simple structured format that can be interpreted not only by JavaScript libraries but by almost any language. It's also human-readable.

There are various Python libraries designed to interpret and display JSON data. With JSON-based libraries, the data is fully contained in a JSON data file. This makes it possible to integrate plots with various visualization tools and techniques.

## 1.4 WebGL-based Libraries

The WebGL standard is a graphics standard that enables interactivity for 3D plots. Much like how HTML5 made interactivity for 2D plots possible (and plotting libraries were developed as a result), the WebGL standard gave rise to 3D interactive plotting libraries.

Python has several plotting libraries that are focused on the development of WebGL plots. Most of these 3D plotting libraries allow for easy integration and sharing via Jupyter notebooks and remote manipulation through the web.

## 1.5 Other Libraries

There are also a variety of other Python plotting libraries, many of which create Python wrappers for other languages and visualization platforms.

# 2 Popular Python Data Visualization Libraries

This book will cover the most popular data visualization libraries for Python, which fall into the five different categories defined above. The libraries covered in this book are: *Matplotlib*, *Pandas*, *Seaborn*, *Bokeh*, *Plotly*, *Altair*, *GGPlot*, *GeoPandas*, and *VisPy*.

You'll need to know what these different libraries are capable of, in order to choose the proper library for your project's needs. Let's take a quick look at these different libraries, some of their unique distinctive features, and what they're used for.

## 2.1 Matplotlib-based Python Libraries

### 2.1.1 Matplotlib

As already stated above, Matplotlib is one of the most common and widely used visualization libraries, used to create static 2D plots, although it does have some support for 3D visualizations. Matplotlib is structured in a fashion that allows the user to create and customize multiple plots for a single image, achieved through the creation of subplots. It's intended to make producing both simple and advanced plots straightforward and intuitive and has support for both static and interactive visualization modes. Though, it's relatively limited when it comes to interactive visualization.

Matplotlib is able to generate numerous different plot types and styles, and it can work along with general-purpose Python GUI libraries like *Qt* and *Tkinter*.

### 2.1.2 Pandas

*Pandas* is a data analysis and manipulation library. While Pandas does come with some visualization and plotting functions, the main reason Pandas is so popular and widely used is that the library makes manipulating data simple and straightforward. Pandas can read data in many different formats, and it creates a Python data object filled with rows and columns, called a *DataFrame*.

These rows and columns are easy to manipulate through built-in functions that let the user merge, split, view, filter, sort, and otherwise alter the data within them, all done with relatively simple commands.

For these reasons, Pandas is frequently used alongside the other data visualization libraries - to prepare the data in question for analysis.

### 2.1.3 Seaborn

*Seaborn* is a visualization library that adds onto Matplotlib's basic functions. Seaborn is intended to enable the easy creation of informative and attractive visualizations. Seaborn gives the user more control over their plots, letting them do things that aren't possible with normal Matplotlib.

This includes the ability to easily produce less common types of visualizations such as heatmaps, violin plots, and joint plots, amongst other plots. Seaborn's goal is to abstract away many of Matplotlib's low-level functions and methods, letting the user create visually impressive plots with less code compared to Matplotlib.

Seaborn gives you more customization options for your plots as well, allowing you to use preset themes or customize the plots to your liking. It also enables efficient handling of dataframes and time-series data.

### 2.1.4 GeoPandas

*GeoPandas* is an extension to the Pandas plotting library designed to make it easier to work with geospatial/geographical data. GeoPandas enables the types of data manipulation possible in Pandas on geometric data, letting you easily carry out visualization tasks that would typically require a spatial database.

GeoPandas allows you to specify the shape of graph regions using special shapefiles, and to clip points and lines to the boundary mask.

## 2.2 JavaScript-based Libraries

### 2.2.1 Bokeh

*Bokeh* is a visualization library that allows the user to create interactive visualizations that can be displayed in Jupyter notebooks and web browsers. Bokeh is focused on the production of highly interactive visualizations, unlike Matplotlib which has just a handful of interactive options. Visualizations in Bokeh are based around objects called “glyphs”, which you can render in numerous different shapes and styles.

Bokeh lets you choose different tools to include alongside your visualization. These tools let you select groups of data points, hover over points to see more information about them, zoom in on multiple graphs at once, and more.

It also allows you to construct numerous different plots with various styles, all the while maintaining high performance across large datasets. Bokeh supports HTML formatting and exporting and has native Pandas integration, allowing you to edit dataframes and the resulting visualizations easily.

With Bokeh, it’s easy to create a well-styled interactive HTML file which you can then embed into a page or presentation.

### 2.2.2 Plotly

Like Bokeh, *Plotly is designed specifically* with the purpose of creating interactive plots. Plotly supports numerous use cases like statistical, geographic, scientific, and even 3D datasets. Similar to Bokeh’s use of glyphs, the fundamental unit of a Plotly plot is the “trace”. You can combine multiple traces and display them all on a single figure.

Plotly for Python is based on JavaScript’s Plotly library and it can be used to create more than 40 different types of plots and charts, each of which can be displayed in a Jupyter notebook or saved in an HTML file. Plotly allows the user to save their plots in the cloud or as a file on their device.

Plotly plots are interactive by default, and they can be created with JSON charts as well as easily embedded in web pages. You can also export Plotly graphs in a variety of different formats, such as PNG, SVG, PDF, and HTML to your local machine.

## 2.3 JSON-based Libraries

### 2.3.1 Altair

*Altair* is a Python library designed explicitly for the visualization of statistical data. Altair is based on the Vega and Vega-Lite standards, meaning that you use *visualization grammar* (specific phrases) that allow you to specify the level of interactivity and style you want your graph to have. Vega specifications are used to define how interactive visualizations are created in *JavaScript Object Notation* (JSON). Altair is a declarative library, and all you need to do is declare which kind of graph you’d like to create along with some desired features for it.

With Altair, you can produce effective visualizations with minimal code. You can often create complex plots with just a single line of code. However, Altair does lack some of the more advanced customization features of the other libraries.

Altair is designed to quickly create interactive statistical visualizations that can be integrated with *IPython* notebooks. Altair also lets you create compound charts comprised of different layers.

## 2.4 WebGL-Based

### 2.4.1 VisPy

*VisPy* is a 2D and 3D visualization library, created primarily to assist in the visualization of big data. Unlike the other libraries mentioned here, VisPy makes use of *Graphics Processing Units* (GPUs) to display the visualization of large datasets.

VisPy supports visualizations of scientific and statistical plots featuring millions of data points. It's intended to be scalable, easy to use, and fast. With having both low-level and high-level interfaces, VisPy makes it possible to create visualizations with relatively few lines of code and then edit those visualizations to your needed specifications.

It has OpenGL support, on which it currently bases some of its functionality, though it does require knowledge of *the OpenGL Shaders Language* (GLSL) to use.

## 2.5 Other

### 2.5.1 GGplot

*GGplot* is intended to make producing plots simple and efficient, rendering them with minimal code. It uses the “*Grammar of Graphics*” standard, borrowed from R. GGplot graphs contain consistent basic elements, which makes graphs uniform and easy to read.

GGplot lets you perform aesthetics mapping, meaning that you can control how variables within your dataset are mapped onto visual properties, defining mappings for different variables and layers of your graph.

## 3 1. Matplotlib

*Matplotlib* is the most widely used data visualization and plotting library in all of Python. In fact, as we've said before, many of the other libraries in this book utilize attributes of Matplotlib to display the plots they generate.

Much of Matplotlib's popularity comes from the fact that it is highly customizable, with users able to edit almost every aspect of a Matplotlib plot.

Matplotlib plots are comprised of a *hierarchy of objects*. At the top level of the plot, the **Figure** is what contains the rest of the plot elements. The intermediate and lower level plot elements are objects and elements like the Axes, Labels, Ticks, and Legends. All of these elements can be tweaked by the user.

In this section, we'll cover the features of Matplotlib, and when you would want to use it. We'll then move on to covering the layout and elements that comprise a Matplotlib plot, demonstrating how to customize these elements.

We'll then go over some examples of the visualizations that you can create with Matplotlib.

### 3.1 Features of Matplotlib

One reason for Matplotlib's enduring popularity is the fact that every element of a Matplotlib plot can be customized. Plots in Matplotlib are all based on **Figures**. The **Figure** is the whole window which holds a single plot or even multiple plots.

Within the **Figure**, various elements like **Axes**, **Lines**, and **Markers** can be created. Aspects like the size and angle of the plot's ticks, the position of the legend, and the thickness of lines can all be manipulated.

Matplotlib also allows you to create multiple plots within a single figure, with subsequent plots being referred to as subplots.

It offers support for both interactive and static visualization modes. When Matplotlib graphs are rendered as interactive graphs, they have to be displayed with one of a few different graphical user interface platforms like Qt, Tkinter, or WxWidgets.

When the visualization is saved to a drive as a file, the visualization is considered to be a hardcopy backend, which are noninteractive. Matplotlib can render visualizations in various file formats such as *JPG*, *PNG*, *SVG*, and *GIF*. Matplotlib is best used for exploratory data analysis and for producing static plots for scientific publications. Matplotlib's core of features lets you quickly explore data for interesting patterns and render simple, static visualizations for reports.

However, if you need to produce interactive visualizations, visualize big data, or produce plots for inclusion in graphical user interfaces, you may be better off using one of the other libraries covered in this book.

Matplotlib supports both simple and complex visualization options. You can use a series of pre-set options to create visualizations, or you can create your own figures and axes that you can customize to your liking.

### 3.2 Anatomy and Customization of a Matplotlib Plot

As previously mentioned, one of Matplotlib's most loved features is that it lets the user customize just about every aspect of the plots it generates. It's important to understand how Matplotlib plots are constructed so that you can edit them to your liking.

For that reason, we'll spend some time covering the anatomy and structure of a Matplotlib plot:

- **Figure** - The figure is what contains all of the other elements of the plot. You can think of it as the canvas that all of the elements of the plot are painted on.
- **Axes** - Plots have X and Y axes, with one variable located on the X-axis and one variable on the Y-axis.
- **Title** - The title is the description given to the plot.
- **Legend** - contains information regarding what the various symbols within the plot represent.
- **Ticks** - Ticks are small lines used to point to different regions of the graph, mark specific items, or delineate different thresholds. For example, if the X-axis of a graph contains the values 0 to 100, ticks may show up at 0, 20, 40, 60, 80, and 100. Ticks run along the sides, as well as the bottom, of the graph.
- **Grids** - Grids are lines in the plot's background that make it easier to distinguish where different values on the X and Y axes intersect.
- **Lines/Markers** - Lines and markers are what represent the actual data within a plot. Lines are typically used to graph continuous values, while markers/points are used to graph discrete values.

Now that we've covered the elements of a Matplotlib plot, let's take some time to examine how you can customize these different attributes and components.

### 3.3 Plotting and Plot Customization

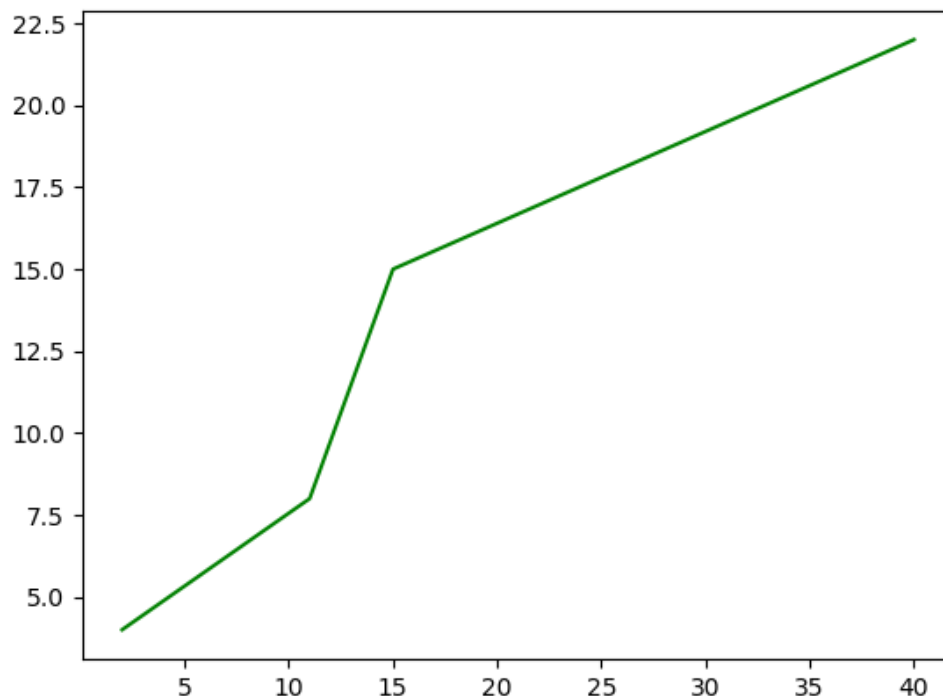
#### 3.3.1 Creating a Plot and Figure

Plotting in Matplotlib is done with the use of the PyPlot interface, which has MATLAB-like commands. You can create visualizations with either a series of presets (the standard way), or you can create figures and axes to plot your data on yourself. We'll cover the simple way of creating plots first and then we'll go into how you can create customizable plots.

PyPlot allows the user to quickly generate professional, standardized plots with just a few lines of code.

First, we'll import matplotlib and the pyplot module. After importing the PyPlot module, it's very simple to call any one of a number of different plotting functions and pass the data you want to visualize into the desired plot function. Then we'll create a simple plot with some random numbers. When we create plots in Matplotlib, the first set of values are those on the X-axis, while the second set of numbers is the Y-axis values. It is possible to plot with just the X-axis values, as Matplotlib will use default values for the Y-axis. You can also pass in a color for the lines:

```
1 import matplotlib.pyplot as plt
2 plt.plot([2, 11, 15, 40], [4, 8, 15, 22], color='g')
3 plt.show()
```



The `plot()` function actually constructs the plot with its elements. The `show()` function is what displays the plot to us when we run the code.

Pyplot mimics aspects of MATLAB's plotting style, meaning that you can style the plot with a series of style commands. One of the style commands is color, which we saw above.

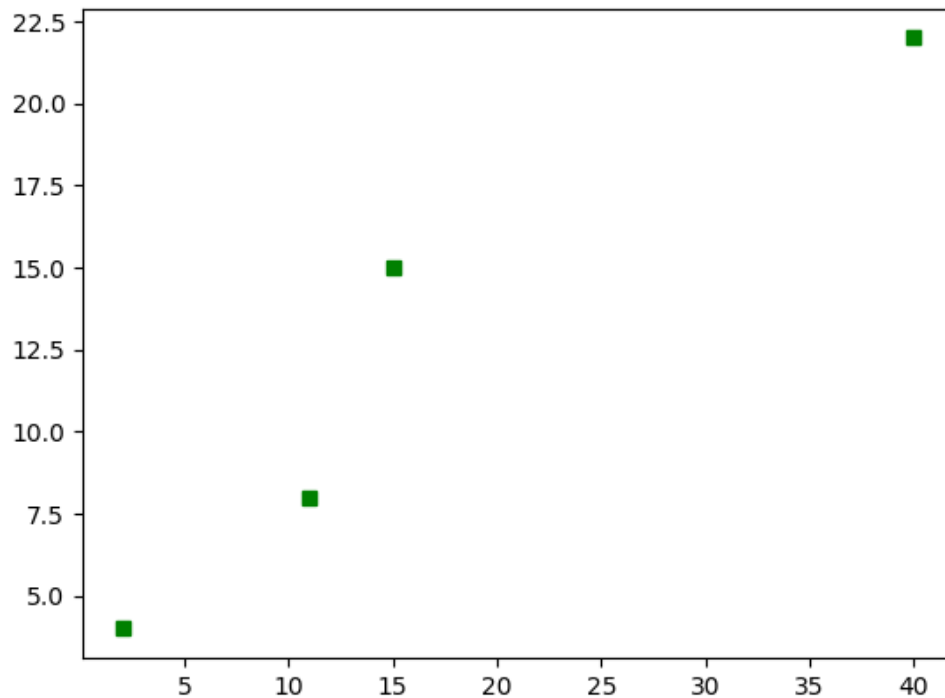
You can also change the symbols used to plot the variables. By default, a solid line is drawn, but you can select other symbols like circles, squares, or triangles.

You can pass the color and symbol instructions in as the third argument of the call to construct the plot. You can view some of the various options for plotting symbols [here](#)<sup>2</sup>.

You can use `--` to create dashes, `s` for squares, or `^` for triangles. For colors, you can use `r` for red, `b` for blue, and `g` for green.

Here's how we could create a plot with green squares:

```
1 plt.plot([2, 11, 15, 40], [4, 8, 15, 22], 'gs')
2 plt.show()
```



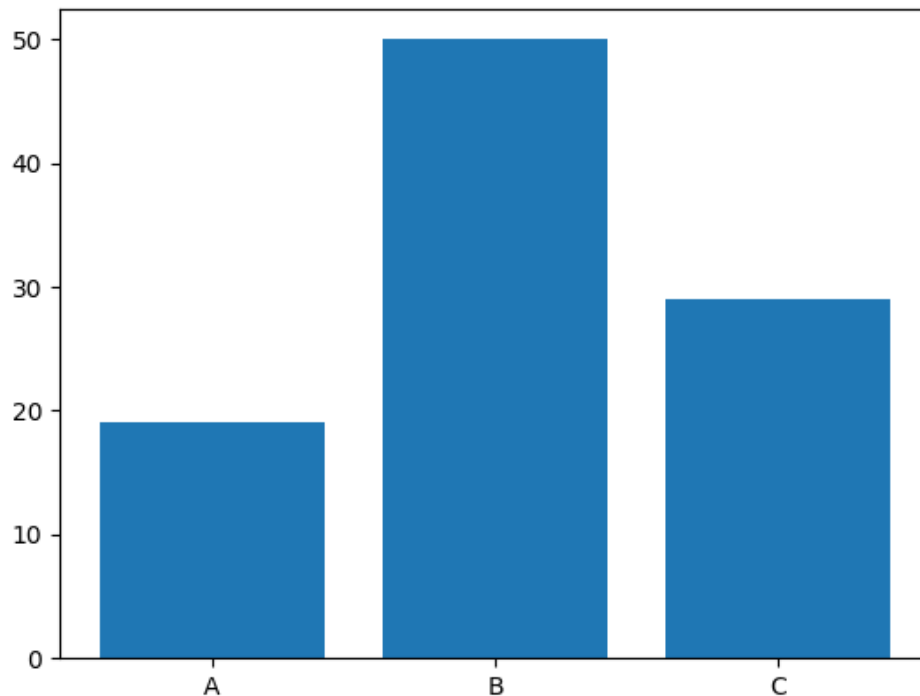
The plots we made above were continuous variables, now we'll explore how to create plots using categorical variables.

You can plot categorical variables by specifying the different categories and values in the form of lists and then passing those variables to the adequate plotting function. For example, bar charts are commonly used for categorical values.

Let's create and plot a bar chart:



```
1 names = ['A', 'B', 'C']
2 values = [19, 50, 29]
3
4 plt.bar(names, values)
5 plt.show()
```



There's an alternate way of creating plots with Matplotlib. The method above allows you to quickly create plots, but if you want more control over how the plot is created, you can create a **Figure** object yourself.

Without creating a **Figure** object, Matplotlib creates a default one for you, with the default settings. To change them, you can use the **figure()** function of the **pyplot** module to create a figure and then specify some properties. For example, you can set the dimensions of the figure you want to create. The dimensions are passed in using a list with four values between 0 and 1.

The four numbers specify the dimensions in this order: left, bottom, width, height. You can also do this with the **add\_subplot()** function, discussed below.

Let's create a figure and add some information regarding the axes.

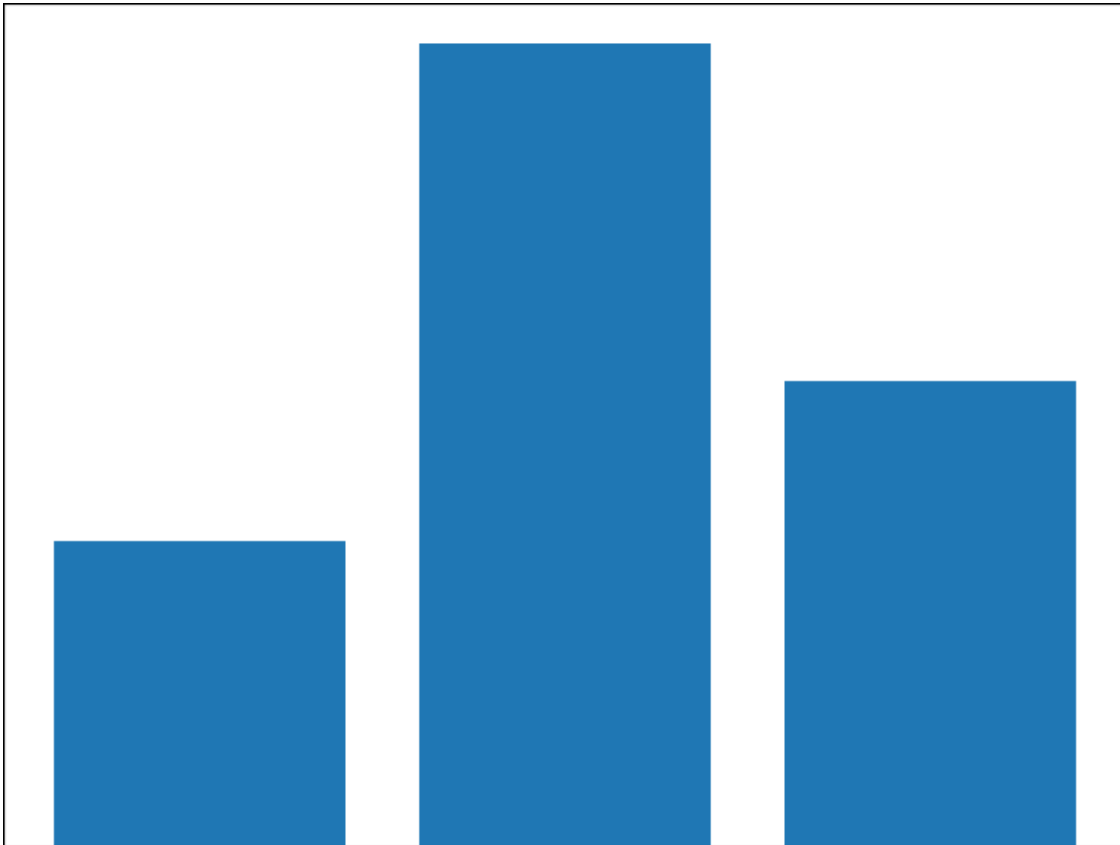
### 3.3.2 Axes

**Axes** objects sit within the figure you have created. Creating an axes object will give you greater control over how data is visualized and other elements of the plot are created.

When using an axes object, you can control how individual subplots are displayed on those axes. You can think of it like this: Figures hold axes and every axes object can store its own plots. The **Axes** instance will contain most of the elements of a figure and you can have multiple **Axes** for a single figure.

These elements include ticks, lines, text, polygons, etc. We'll explore how to change these elements throughout the Customizing a Plot section, up ahead. For now, let's just create an axes object on a figure:

```
1 fig = plt.figure()
2 ax = fig.add_axes([0, 0, 1, 1])
3 names = ['A', 'B', 'C']
4 values = [19, 50, 29]
5 ax.bar(names, values)
6 plt.show()
```



The `fig.add_axes()` function returns a new **Axes** object which we've packed in `ax`. Using this object, we'll be adding elements. For example, we've called `ax.bar()` to plot a bar graph instead of calling `plt.bar()` like before.

`ax` belongs to the `fig` so everything added to the `ax` will also be added to the `fig`. The arguments we've passed to the `add_axes()` function were `[0, 0, 1, 1]`. These are the **left**, **bottom**, **width**, and **height** of the `ax` object.

The numbers are fractions of the figure the **Axes** object belongs to, so we've told it to start at the

bottom-left point (0 for `left` and 0 for `bottom`) and to have the same height and width of the parent figure (1 for `width` and 1 for `height`).

We can't really see the `ax` at this point, other than the plot is missing some elements as opposed to the previous example where they were set to default.

You can also delete axes through the use of the `delaxes()` function:

```
1 fig.delaxes(ax)
```

Now that we know the general method for creating plots in Matplotlib, let's take a look at the many options you have at your disposal for customizing these plots.

### 3.3.3 Subplots

Matplotlib allows you to create multiple plots within the same figure. In order to add multiple plots, you need to create a "subplot" for each plot in the figure you'd like to use.

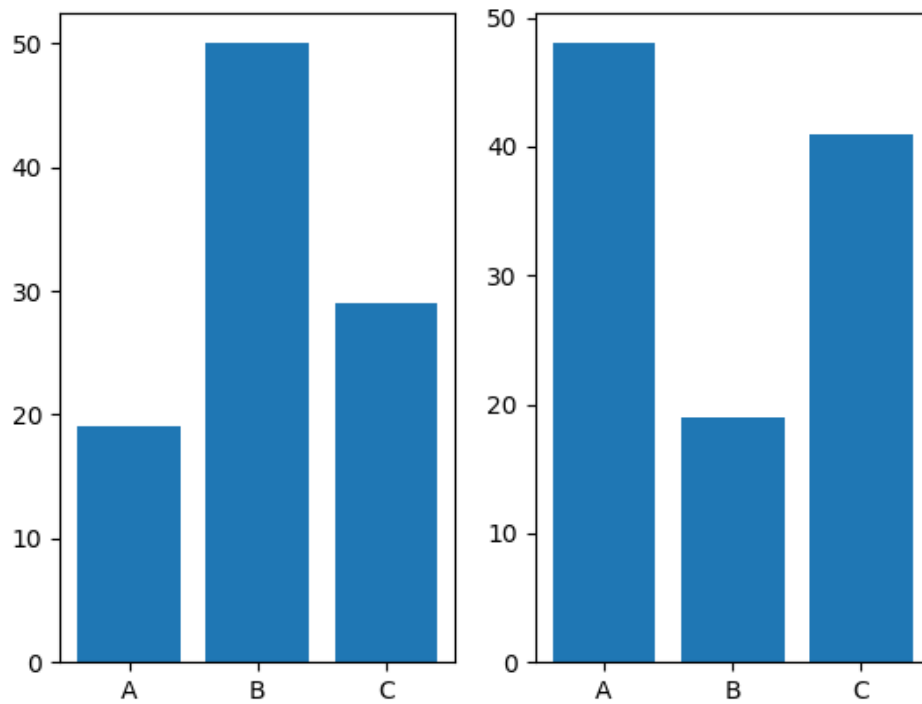
This is done with the `add_subplot()` function, which accepts a series of numeric arguments.

The first number specifies how many rows you want to add to the figure, the second number specifies how many columns you want to add, and the third number specifies the number of the plot that you want to add.

This means that if you in passed in 111 into the `add_subplots()` function, one new subplot would be added to the figure. Meanwhile, if you used the numbers 221, the resulting plot would have four axes with two columns and two rows - and the subplot you're forming is in the 1st position.

Here's how we would create two subplots in the same figure, notice that we have created two axes objects:

```
1 fig = plt.figure()
2
3 names = ['A', 'B', 'C']
4 values = [19, 50, 29]
5 values_2 = [48, 19, 41]
6
7 ax = fig.add_subplot(121)
8 ax2 = fig.add_subplot(122)
9
10 ax.bar(names, values)
11 ax2.bar(names, values_2)
12 plt.show()
```



We've created two subplots in a figure with 1 row and 2 columns. They're sitting side by side. If we had created a figure with 2 rows and 1 column:

```
1 fig = plt.figure()
2
3 names = ['A', 'B', 'C']
4 values = [19, 50, 29]
5 values_2 = [48, 19, 41]
6
7 ax = fig.add_subplot(211)
8 ax2 = fig.add_subplot(212)
9
10 ax.bar(names, values)
11 ax2.bar(names, values_2)
12 plt.show()
```

We'd be looking at something like this:

